

JavaScript Cookbook

A Practical Guide for Solving Common JavaScript Problems

Author: Luv, Zofia

Introduction

JavaScript is one of the most widely used programming languages in modern web development. It allows developers to build interactive websites, dynamic web applications, mobile apps, and even backend systems using technologies like Node.js.

This mini ebook inspired by **JavaScript Cookbook** presents simple and practical recipes for solving common programming challenges in JavaScript. Each chapter includes clear explanations and real-world examples to help both beginners and experienced developers improve their coding skills.

Chapter 1 — Data Manipulation

Data manipulation is one of the most important tasks in JavaScript development.

Transforming an Array

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(number => number * 2);

console.log(doubled);
```

Output

```
[2, 4, 6, 8]
```

Explanation

The `map()` method creates a new array by applying a transformation to each element.

Filtering Data

```
const ages = [12, 18, 25, 14, 30];

const adults = ages.filter(age => age >= 18);

console.log(adults);
```

Output

[18, 25, 30]

Explanation

The `filter()` method returns only the elements that match a specific condition.

Chapter 2 — DOM Manipulation

The DOM (Document Object Model) represents the structure of an HTML page.

Changing HTML Content

```
<p id="message">Hello</p>
```

```
document.getElementById("message").textContent = "Welcome to JavaScript";
```

Explanation

`getElementById()` selects an HTML element and allows you to modify its content.

Handling Click Events

```
<button id="btn">Click Here</button>
```

```
document.getElementById("btn").addEventListener("click", () => {  
    alert("Button clicked!");  
});
```

Explanation

`addEventListener()` listens for events such as clicks, keyboard input, or mouse movements.

Chapter 3 — Asynchronous Programming

Asynchronous programming allows applications to perform tasks without blocking execution.

Using `fetch()`

```
fetch("https://jsonplaceholder.typicode.com/users")  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.log(error));
```

Explanation

`fetch()` retrieves data from an external API.

Using `async/await`

```
async function loadUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    const data = await response.json();

    console.log(data);
  } catch(error) {
    console.log(error);
  }
}

loadUsers();
```

Advantage

`async/await` makes asynchronous code easier to read and maintain.

Chapter 4 — API Integration

APIs allow applications to communicate with external services.

Sending Data with POST

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    title: "My Article",
    body: "Article content"
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

Explanation

- `method: "POST"` sends data to the server
- `headers` define the content type
- `body` contains the data being sent

Chapter 5 — Performance Optimization

Good developers also focus on application performance.

Avoiding Repeated DOM Searches

Bad Practice

```
for(let i = 0; i < 1000; i++) {  
    document.getElementById("title").style.color = "red";  
}
```

Good Practice

```
const title = document.getElementById("title");  
  
for(let i = 0; i < 1000; i++) {  
    title.style.color = "red";  
}
```

Why?

This avoids repeatedly searching for the same DOM element, improving performance.

Chapter 6 — JavaScript Best Practices

Using const and let

```
const name = "Chrislain";  
let age = 25;
```

- const → for values that do not change
- let → for values that can change

Using Clear Variable Names

Poor Example

```
let x = 5;
```

Better Example

```
let studentCount = 5;
```

Clear variable names make code easier to understand and maintain.

Conclusion

JavaScript is a powerful language for building modern and interactive applications. With these practical recipes, developers can quickly solve common problems and improve their productivity.

The key to mastering JavaScript is consistent practice:

- write code every day,
- build small projects,
- learn from mistakes,
- explore new solutions.

Over time, JavaScript development becomes more natural and efficient.

Practice Exercises

1. Build a simple calculator in JavaScript.
2. Consume a weather API using `fetch()`.
3. Create a Todo List application.
4. Build an interactive counter.
5. Display books dynamically from a Spring Boot API.